

Constrained Object Detection & Reasoning API — Technical Memo

RAP Hackathon submission — RT-DETR-L electrical-equipment detection (Part A) with a deterministic natural-language reasoning layer (Part B)

1. Problem & Domain

Constrained object-detection-and-reasoning API for electrical equipment inspection imagery. Part A fine-tunes RT-DETR-L on 8 non-COCO classes (arc, disconnecter, disconnecter_open, insulator, spark, switch, switchgear, transformer). Part B is a hand-written (no agentic-framework) reasoning layer that answers questions over detections, returning "insufficient information" rather than guessing when unsupported.

2. Dataset Source & Preparation

Roboflow project "Station" v2 (workspace ppe-detection-7kco8, slug station-raaga-ogkgf), License **CC BY 4.0**, exported 2025-09-23, 9,718 source images (COCO format). Root category helmet-insulator (id 0, unannotated supercategory) is excluded via CATEGORY_MAP. Pipeline: raw train/valid/test (COCO) → convert_coco.py → yolo_dataset/ (YOLO format, the format actually trained on). yolo_dataset/data.yaml uses a relative dataset root (path: yolo_dataset) and is portable across machines.

3. Split Strategy & Integrity Checks

Splits inherited from the Roboflow export. integrity.py cross-checks images on disk vs. COCO references and does MD5-based duplicate/cross-split-leak detection; audit.py reports per-class counts and flags invalid boxes; dataset_stats.py reports dimension/bbox-size distributions. The full held-out test split (568 images) was used for the final metrics. After those metrics were locked, 50 images from the same test split were used only for failure-case analysis — never for training, tuning, or threshold selection.

4. RT-DETR-L Training Setup

- Config:** epochs=30, imgsz=640, batch=8, device=0, seed=0, deterministic=False, save_period=1.
- An earlier 50-epoch run on Colab (T4) was interrupted by a GPU usage-limit disconnect at epoch 28/50; training was restarted and completed on Kaggle with the config above.
- Training environment:** NVIDIA T4, Python 3.12.13, PyTorch 2.10.0+cu128, Ultralytics 8.4.146.
- Training wall-clock time was not recorded.**

5. Final Test-Set Metrics

568 images, 1,934 instances, model.val() on weights/best.pt:

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	568	1934	0.916	0.896	0.933	0.669
arc	76	119	0.918	0.924	0.958	0.567
disconnecter	65	171	0.918	0.922	0.983	0.783
disconnecter_open	62	144	0.927	0.888	0.951	0.717
insulator	333	1023	0.912	0.896	0.929	0.619
spark	67	211	0.854	0.668	0.763	0.378
switch	78	162	0.871	0.914	0.938	0.700
switchgear	12	29	0.937	0.966	0.946	0.715
transformer	61	75	0.987	0.990	0.995	0.876

Evaluation environment (distinct from training): Ultralytics 8.4.147, Python 3.12.2, torch 2.8.0+cpu, local CPU. spark is the weakest class (recall 0.668, mAP50-95 0.378).

6. Reasoning Layer & API

Hand-written control flow only — no LangChain/LangGraph/CrewAI/AutoGen or any agentic framework. Intents: PRESENCE, COUNT, LOCATION, RISK_ASSESSMENT, LIST_ALL, OUT_OF_SCOPE. Detections below CONF_THRESH=0.5 are excluded; [0.5, 0.6) are reported but hedged. HIGH_RISK_CLASSES={arc, spark}, MEDIUM_RISK_CLASSES={disconnecter_open}. Out-of-taxonomy questions route to OUT_OF_SCOPE.

Endpoints: GET /health; POST /detect (image upload → boxes/classes/confidences; 400 on non-image upload, 500 with a clean body on unexpected errors); POST /ask (question + detections → intent, answer_state, answer).

7. Five Failure Cases

1. **Small-object recall failure** — **spark**. Several small spark instances (sub-1% image area) in cluttered frames are missed or weakly localized at inference confidence thresholds — consistent with spark having the lowest recall (0.668) of any class.
2. **Class confusion** — **spark** → **arc** (image 8306b575-images_9...jpg). A spark ground-truth box is predicted as arc at IoU 0.694, confidence 0.669. *Likely cause*: arc and spark are visually similar bright, localized light phenomena, making this plausibly a class-boundary ambiguity rather than a random error — not independently verified beyond this single case.
3. **Class confusion + fragmentation** — **switch** → **multiple insulators**. A switch region is fragmented into several smaller insulator predictions instead of one correct switch box. *Plausible explanation*: sub-components of a switch assembly may visually resemble individual insulator units; this mechanism is not directly confirmed.
4. **Complete class-wide miss** — **insulator** (image 3dcc4461-images_20...jpg). Three real insulator instances (3–5% image area each) receive zero insulator predictions (`best_iou_sameclass=0.0`); the model's only output in that region is one large switch detection (conf 0.927) overlapping all three. *Root cause*: **not conclusively determined** — feature competition from a dominant co-located object is a plausible explanation, unconfirmed without further inspection.
5. **Duplicate/over-detection in dense clutter** — **insulator**. 10 real instances, 11 predictions ≥ 0.5 confidence: 10 correct plus one spurious false positive (confidence 0.748, IoU 0.007 vs. nearest real instance). *Likely cause*: dense, visually repetitive same-class clutter producing an extra, spurious detection rather than a hallucinated class.

8. Limitations

- Training wall-clock time not recorded; only the resulting checkpoint (`weights/best.pt`) was retrieved from Kaggle.
- Evaluation ran in a different (CPU) environment than training (GPU) — metrics are real but not a training-throughput benchmark.
- spark is the weakest class; Cases 1–2 suggest a genuine visual-ambiguity limit, not confirmed as a data bug.
- Case 4's root cause is explicitly unconfirmed.
- No person/PPE classes exist in the taxonomy; such questions correctly route to `OUT_OF_SCOPE`.
- A hidden hold-out set grades Part A independently; self-reported test-set metrics are necessarily incomplete evidence of generalization.

9. Reproducibility

1. Raw Roboflow export → `convert_coco.py` → `yolo_dataset/` (regenerate rather than hand-copy `data.yaml`).
2. Run `integrity.py` and `audit.py` against the raw export before trusting a conversion.
3. Train RT-DETR-L via Ultralytics using the Section 4 config.
4. Evaluate with `model.val(data='yolo_dataset/data.yaml', split='test')`.
5. `pip install -r requirements.txt`; `pytest app/tests/ part_b/tests/` (27 tests, all passing); serve with `uvicorn app.main:app`.

Docker: a Dockerfile is provided (CPU-only PyTorch wheels via `--index-url https://download.pytorch.org/whl/cpu`, avoiding an unnecessary CUDA-runtime install) and has been reviewed, but **has not been built or run locally** — no Docker installation was available in this development environment, so end-to-end container behavior is unverified.